

Instrucciones

1. **Valor: 40%**
2. Fecha máxima de entrega de la última parte de la tarea, que incluye todas las etapas anteriores: martes 2 de junio 2026 antes de las 11:55 pm.
3. Trabajo grupal
4. Seleccionar el repositorio con el código fuente de una aplicación cliente-servidor deliberadamente vulnerable.
5. La tarea se entregará por etapas.

Objetivos de aprendizaje

1. Aprender a redactar requerimientos de seguridad y Threat Modeling
2. Reconocer vulnerabilidades en código fuente tanto en componentes servidor como cliente.
3. Localizar puntos de inyección, fallos en controles de acceso y vectores que permiten escalación de privilegios.
4. Corregir código aplicando principios secure-by-design y demostrar la corrección mediante pruebas automatizadas.
5. Implementar técnicas de trazabilidad y logging que faciliten la detección de intrusiones y reduzcan riesgos de audit poisoning.
6. Documentar y justificar las decisiones técnicas y de diseño de manera profesional.

Parte 1 – Requerimientos de seguridad y Threat Modeling

Instrucciones:

1. Seleccionar una aplicación que hayan programado en otro curso, que tengan el código fuente subido en github y que al menos alguno de los miembros del grupo haya participado en el desarrollo y levantamiento de requerimientos. Debe ser una aplicación web, también pueden utilizar alguna aplicación pública.
2. Esta aplicación se utilizará durante todo el curso para remediaciones de software seguro
3. Realizar la etapa de Threat Modeling para dicha aplicación, siguiendo los mismos pasos del ejercicio Quiz Grupal 1, deben incluir 12 amenazas (3 por cada miembro del equipo).

4. Redactar 12 requerimientos de seguridad de la aplicación, 3 requerimientos por cada miembro de equipo
5. Valor 10% del proyecto final
6. Fecha de entrega: 24 de marzo 2026 antes de 11:45 pm (pueden utilizar la clase asincrónica para terminar detalles del proyecto)

Entregable:

- Un documento corto (10 páginas máximo, incluyendo la portada, sin incluir anexos) que incluya:
 1. Descripción del sistema
 2. Lista de activos de todo el proyecto
 3. Diagrama de flujo de datos de todo el proyecto
 4. Las amenazas y riesgos identificados para todo el proyecto (Threat Modeling)
 5. Las mitigaciones propuestas para las amenazas identificadas.
 6. El listado detallado de requerimientos de seguridad con el formato visto en clase
 7. Reflexión final sobre las amenazas y riesgos identificados para el proyecto
- 1 único video grabado máximo 1 minuto cada persona del grupo, con la explicación de cada una de las partes que realizó dentro del proyecto, debe utilizar material visual para su presentación

Rúbrica de evaluación:

Rúbrica de Evaluación – Proyecto de Threat Modeling

Escala de evaluación

- 4 – Excelente: Cumple completamente, nivel profesional
- 3 – Bueno: Cumple con calidad, pequeños vacíos
- 2 – Básico: Cumple parcialmente, falta profundidad
- 1 – Insuficiente: Incompleto o incorrecto

Documento escrito (70%) / 32 puntos en TOTAL

1. Descripción del sistema

- 4: Explica claramente el sistema, actores, alcance, tecnologías y contexto de uso.
- 3: Descripción clara pero con poco detalle técnico.
- 2: General, faltan elementos clave (actores, alcance, etc.).
- 1: Confusa o incompleta.

2. Identificación de activos

- 4: Lista completa (datos, sistemas, usuarios, infraestructura), bien clasificada y justificada.
- 3: Buena lista, pero con poca clasificación o priorización.
- 2: Activos incompletos o poco claros.
- 1: Identificación incorrecta o muy limitada.

3. Diagrama de flujo de datos (DFD)

- 4: Completo, claro, correcto (procesos, entidades externas, flujos, almacenes de datos).
- 3: Correcto pero con leves omisiones o errores.
- 2: Entendible pero incompleto o desordenado.
- 1: Incorrecto o ausente.

4. Identificación de amenazas y riesgos

- 4: Uso adecuado de metodología (ej. STRIDE), amenazas bien justificadas y contextualizadas.
- 3: Buen análisis pero con poca profundidad o justificación.
- 2: Amenazas genéricas o poco alineadas al sistema.
- 1: Identificación débil o incorrecta.

5. Mitigaciones propuestas

- 4: Mitigaciones claras, viables y alineadas a cada amenaza.
- 3: Mitigaciones adecuadas pero no siempre bien vinculadas.
- 2: Mitigaciones genéricas o poco aplicables.
- 1: Ausentes o incorrectas.

6.Requerimientos de seguridad

- 4: Bien estructurados, medibles, claros (formato correcto).
- 3: Mayormente claros, con pequeños errores de formato.
- 2: Poco claros o mal estructurados.
- 1: Incorrectos o incompletos.

7.Reflexión final

- 4: Análisis crítico, aprendizaje claro, conexión con riesgos reales.
- 3: Buena reflexión pero superficial.
- 2: Reflexión básica.
- 1: Ausente o irrelevante.

8. Calidad del documento (formato y redacción)

- 4: Profesional, bien estructurado, sin errores.
- 3: Buena presentación con pequeños errores.
- 2: Problemas de claridad o redacción.
- 1: Desordenado o difícil de leer.

Video individual (30%) / 16 puntos en TOTAL

1.Claridad de la explicación

- 4: Explicación clara, segura y bien estructurada.
- 3: Clara pero con leves dudas.
- 2: Poco clara o desordenada.
- 1: Difícil de entender.

2. Dominio del tema

- 4: Demuestra dominio total de su parte.
- 3: Buen dominio, con pequeñas imprecisiones.
- 2: Conocimiento limitado.
- 1: No demuestra comprensión.

3. Uso de material visual

4: Apoyo visual claro, relevante y bien integrado.

3: Material útil pero básico.

2: Poco uso o poco relevante.

1: Ausente.

4. Tiempo y presentación

4: Dentro del tiempo, presentación dinámica.

3: Ligeramente fuera de tiempo o poco dinámica.

2: Desbalanceado o apresurado.

1: No cumple con el tiempo.

Resumen de ponderación

Componente	Peso
Documento	70%
Video	30%
Total	100%

Parte 2 – Explotación de Software de Servidores

Fecha entrega Parte 2 y Parte 3: 2 de Junio 2026

Instrucciones:

1. Debe incluirse **2** correcciones de diferente naturaleza por persona del equipo
2. Valor 15%

A. Explotación de Software de Servidores (8 correcciones de diferente naturaleza, 2 por cada miembro del equipo):

- Identificar y corregir un punto de inyección en el servidor (**por ejemplo**: inyección SQL, inyección de comandos o inyección en parámetros).
- Corregir control(es) de acceso para prevenir escalación de privilegios (p. ej. validar roles, chequear objetos por propietario).
- **Aplicar y documentar una defensa** para la “explotación de la confianza” (p. ej. evitar confiar en datos del cliente para decisiones críticas).

Parte 3 – Explotación de Software en Clientes

Instrucciones:

1. Debe incluirse **2** correcciones de diferente naturaleza por persona del equipo
2. Valor 15%

B. Explotación de Software en Clientes (5 correcciones de diferente naturaleza, 1 por cada miembro del equipo):

- Localizar y arreglar una vulnerabilidad relacionada con el cliente como objeto de ataque (p. ej. manejo inseguro de datos recibidos, deserialización insegura).
- Corregir un caso de **XSS** o una exposición de client scripts que pueda permitir ejecución de código malicioso en el cliente.
- Asegurar que el cliente sanea/escapa datos antes de renderizar o ejecutar.

C. Alteración de Entrada de Usuario (2 correcciones en total)

- Añadir o mejorar trazas (tracing) y logging relevante para las rutas vulnerables (con atención a no loguear secretos en claro).
- Implementar pruebas automatizadas que reproduzcan las solicitudes equivalentes que antes explotaban la vulnerabilidad (unit/integration tests).
- Proponer y documentar mecanismos para reducir el riesgo de **audit poisoning** (ej. logs append-only, firma o hashing de logs, envío a un colector remoto).

Workflow y reglas de trabajo para la parte 2, 3

1. Clonar el repositorio original del proyecto
2. Crear un branch local para cada miembro del equipo, donde se identifique el autor, pueden ser las iniciales del nombre.
3. **Commits claros:** Mensajes de commit descriptivos; conservar historial para evaluación.
4. **Pull Request (PR):** Antes de cada entrega final, crear uno o varios PR con las correcciones y pruebas; usar la descripción del PR para explicar cambios y riesgos. **Esta prohibido hacer push directo al main branch, siempre debe pasar por un pull request.**
5. **Tests:** Todas las correcciones deben acompañarse de pruebas que fallen en la versión vulnerable y pasen con la versión corregida. Cada equipo es libre de

presentar el formato de sus pruebas, pero debe haber evidencia de las pruebas antes y después, ya sea como pantallazos con descripción o como casos de prueba individuales

6. **No realizar pruebas fuera del entorno:** Prohibido atacar sistemas ajenos; usar solo el código y dataset seleccionados.

Entregables para la etapa 2 y 3 (formato y contenido)

1. **Repositorio (Git) con:**

- Ramas/commits que muestran el proceso de corrección.
- Document en carpeta tests/ que demuestran la vulnerabilidad y su corrección, es decir, el antes y el después.
- Scripts que reproducen solicitudes equivalentes (curl/Postman collections o scripts) en caso que los usaron.

2. **Informe técnico en formato word editable (máx. 20 páginas incluyendo la portada)** que contenga:

- Portada
- Resumen ejecutivo de lo que trata el proyecto.
- Vulnerabilidades identificadas (archivo/clase/función) y su clasificación (CWE/OWASP cuando aplique).
- Descripción de la corrección aplicada (qué se cambió y por qué).
- Evidencias de las pruebas: logs, salidas de tests, capturas del comportamiento antes/después, con una breve explicación
- Recomendaciones de diseño.
- Declaración de cumplimiento (en la parte final de este documento se indica de que trata).
- Link al video

3. **Video demo (4–10 min):** breve recorrido mostrando la prueba que fallaba en la versión vulnerable y cómo ahora pasa con el parche o la corrección aplicada. Cada estudiante debe aparecer en el video con la cámara encendida explicando su parte en el proyecto. No importa si son videos grabados por separados y posteriormente, pegados manualmente, **únicamente considerar que no debe exceder los 6 minutos en total.**

4. **README.md con pasos de reproducción** (máx. 5 páginas): cómo ejecutar la app vulnerable, ejecutar las pruebas y comprobar las correcciones. Cada estudiante del equipo debe tener una sección exclusiva para describir su aporte del proyecto.

Criterios de evaluación

- **Corrección funcional y pruebas asociadas — 25 pts**
¿Los tests demuestran la vulnerabilidad y su corrección? ¿Las correcciones son reproducibles?
- **Calidad de la corrección/ diseño seguro — 25 pts**
Principios de diseño aplicados (validación centralizada, least privilege, escaping, prepared statements, manejo correcto de sesiones, etc.). Código claro y con buenas prácticas.
- **Análisis y documentación técnica — 15 pts**
Claridad en el informe: identificación precisa del código vulnerable, evidencia, clasificación de vulnerabilidad, y análisis de impacto.
- **Video — 25 pts**
Claridad de la explicación, consistente con el código fuente y el informe.
Participación de todos los miembros del equipo con la cámara encendida y respetando el tiempo total del video.
- **Workflow y buenas prácticas (Git/PR/tests) — 10 pts**
Historial de commits, uso de ramas, descripción de PR.

Recomendaciones opcionales

- Añadir tests que **fallen** en la versión vulnerable: ejemplo, una solicitud maliciosa que provoca X comportamiento. Documenta el payload pero **sin instruir a explotarlo fuera del laboratorio**.
- Preferir correcciones en la capa de servidor para validación e implementación de prepared statements; en cliente, priorizar escapar.
- Para logging: registrar eventos relevantes con contexto suficiente (userID, endpoint, timestamp) pero evitar logs con contraseñas o tokens en texto plano.
- En la documentación del PR, explicar por qué el cambio evita la falla y qué efectos secundarios se consideraron (performance, compatibilidad).

Consideraciones éticas y legales (obligatorio)

- Las pruebas deben limitarse estrictamente al código y entorno facilitados por el curso.
- Incluye en el informe una **Declaración de cumplimiento** firmada por cada miembro del equipo, indicando que todas las pruebas se ejecutaron en el **entorno autorizado de trabajo**.
- No publicar exploits ni publicar datos sensibles derivados del laboratorio.